

test projet

Lorys AGBATCHOSSOU · 2 de septembre de 2026



import et nettoyage des données

```
In [17]:
import yfinance as yf
import pandas as pd

tickers = ["SPY", "TLT", "GLD", "USO", "QQQ"]
data = yf.download(tickers, period="5y", auto_adjust=True)["Close"] #on telecharge Les prix au close sur 5 ans

data

Out [17]:
[*****100%*****] 5 of 5 completed
```

Ticker	GLD	QQQ	SPY	TLT	USO
Date					
2021-09-02	169.250000	369.174133	423.565338	125.360291	48.950001
2021-09-03	171.059998	370.309631	423.462463	124.220215	48.660000
2021-09-07	167.710007	370.833679	421.948395	123.172340	48.040001
2021-09-08	167.289993	369.542938	421.434387	124.010681	48.590000
2021-09-09	168.029999	368.271545	419.630554	125.511208	47.750000
...	...	...	...	...	...
2026-08-26	421.320007	711.369995	766.080017	82.982025	127.349998
2026-08-27	422.600006	721.109985	771.099976	82.812668	130.009995
2026-08-28	408.890015	716.429993	769.349976	82.563622	129.699997
2026-08-31	408.420013	716.760010	767.049988	82.204994	133.699997
2026-09-01	396.750000	707.640015	761.780029	81.870003	141.000000

1254 rows × 5 columns

```
In [18]:
#suppresion des valeurs manquantes, jours fériés différents entre marchés, actifs introduits plus tard
data.isna().sum()

data.shape[0]

#calcul du rendement
returns = data.pct_change()

returns = returns.dropna()

returns

Out [18]:
```

Ticker	GLD	QQQ	SPY	TLT	USO
Date					
2021-09-03	0.010694	0.003076	-0.000243	-0.009094	-0.005924
2021-09-07	-0.019584	0.001415	-0.003575	-0.008436	-0.012741
2021-09-08	-0.002504	-0.003481	-0.001218	0.006806	0.011449
2021-09-09	0.004423	-0.003440	-0.004280	0.012100	-0.017288
2021-09-10	-0.005059	-0.007590	-0.007885	-0.008817	0.021990
...	...	...	...	...	...
2026-08-26	-0.015768	0.000915	0.000222	-0.002037	0.009512
2026-08-27	0.003038	0.013692	0.006553	-0.002041	0.020887
2026-08-28	-0.032442	-0.006490	-0.002269	-0.003007	-0.002384
2026-08-31	-0.001149	0.000461	-0.002990	-0.004344	0.030840

Ticker	GLD	QQQ	SPY	TLT	USO
Date					
2026-09-01	-0.028574	-0.012724	-0.006870	-0.004075	0.054600

1253 rows × 5 columns

Analyse exploratoire EDA

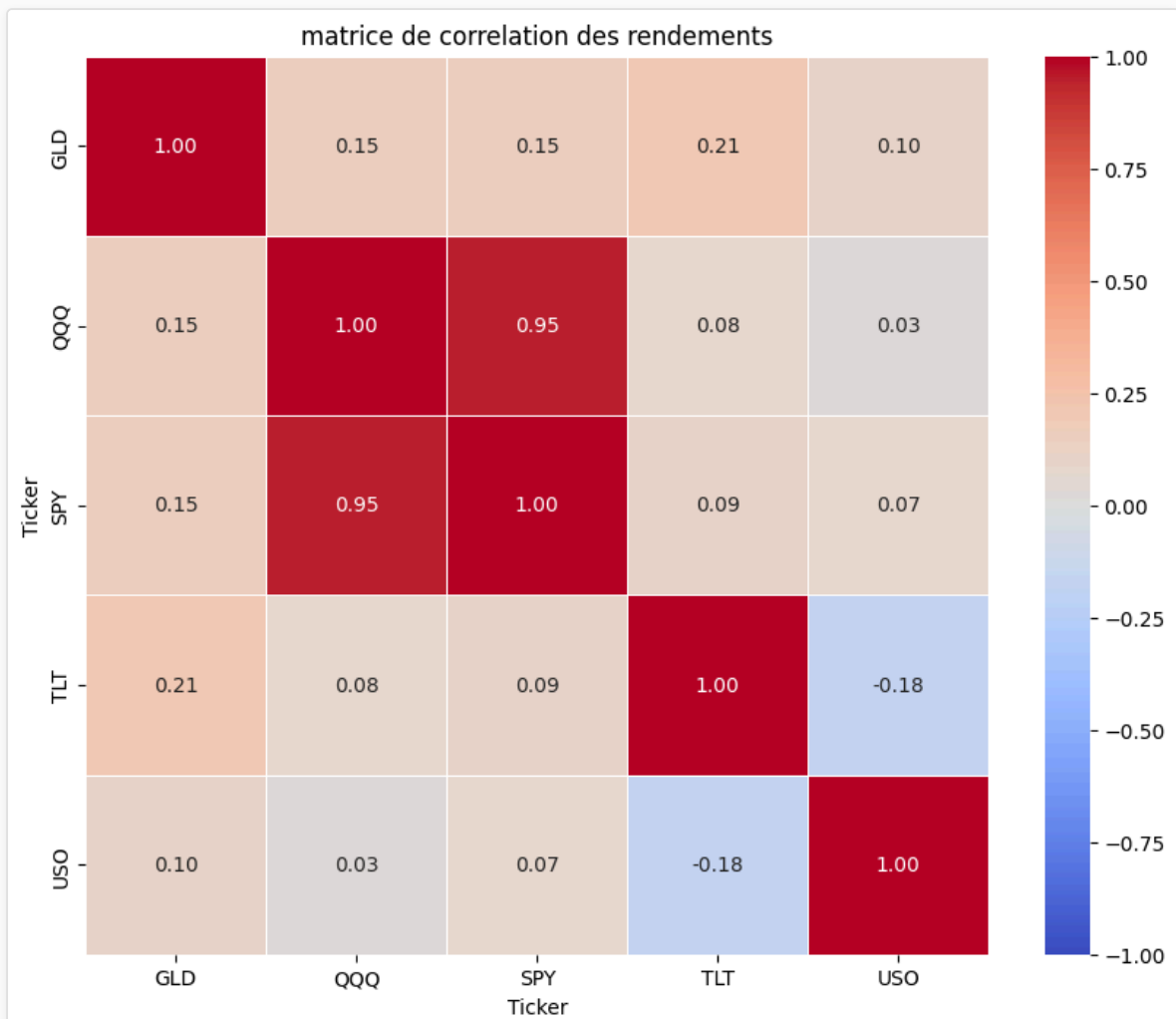
In [19]:

```
#matrice de correlation
import matplotlib.pyplot as plt
import seaborn as sns

corr_returns= returns.corr(numeric_only=True)

#affichage sous forme de heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_returns, annot=True, cmap="coolwarm", vmin = -1, vmax = 1, linewidths=0.5, fmt=".2f")
plt.title("matrice de correlation des rendements")
plt.show()
```

Out [19]:



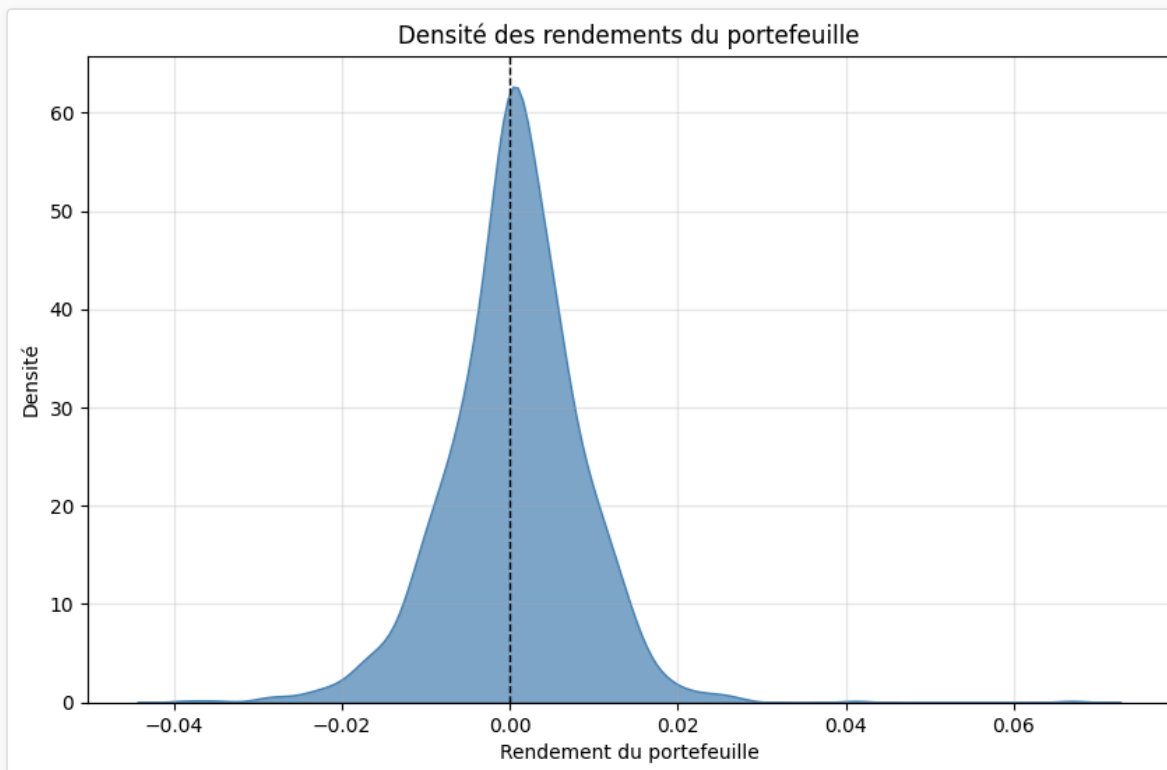
In [20]:

```
#courbe de densité des rendements du portefeuille
import numpy as np
# Portefeuille équipondéré 20%
poids = np.array([1/5, 1/5, 1/5, 1/5, 1/5])
#rendement du portefeuille = somme pondérée des rendements
portfolio_return = returns.dot(poids)
```

```
# Courbe de densité
plt.figure(figsize=(10, 6))
sns.kdeplot(portfolio_return, fill="True", color="steelblue", alpha=0.7)
plt.axvline(0, color="black", linestyle="--", linewidth=1)
plt.title("Densité des rendements du portefeuille")
plt.xlabel("Rendement du portefeuille")
plt.ylabel("Densité")
plt.grid(True, alpha=0.3)
plt.show()

portfolio_return
```

Out [20]:



```
Date
2021-09-03    -0.000298
2021-09-07    -0.008584
2021-09-08     0.002210
2021-09-09    -0.001697
2021-09-10    -0.001472
...
2026-08-26    -0.001431
2026-08-27     0.008426
2026-08-28    -0.009319
2026-08-31     0.004564
2026-09-01     0.000471
Length: 1253, dtype: float64
```

In [21]:

```
from scipy import stats

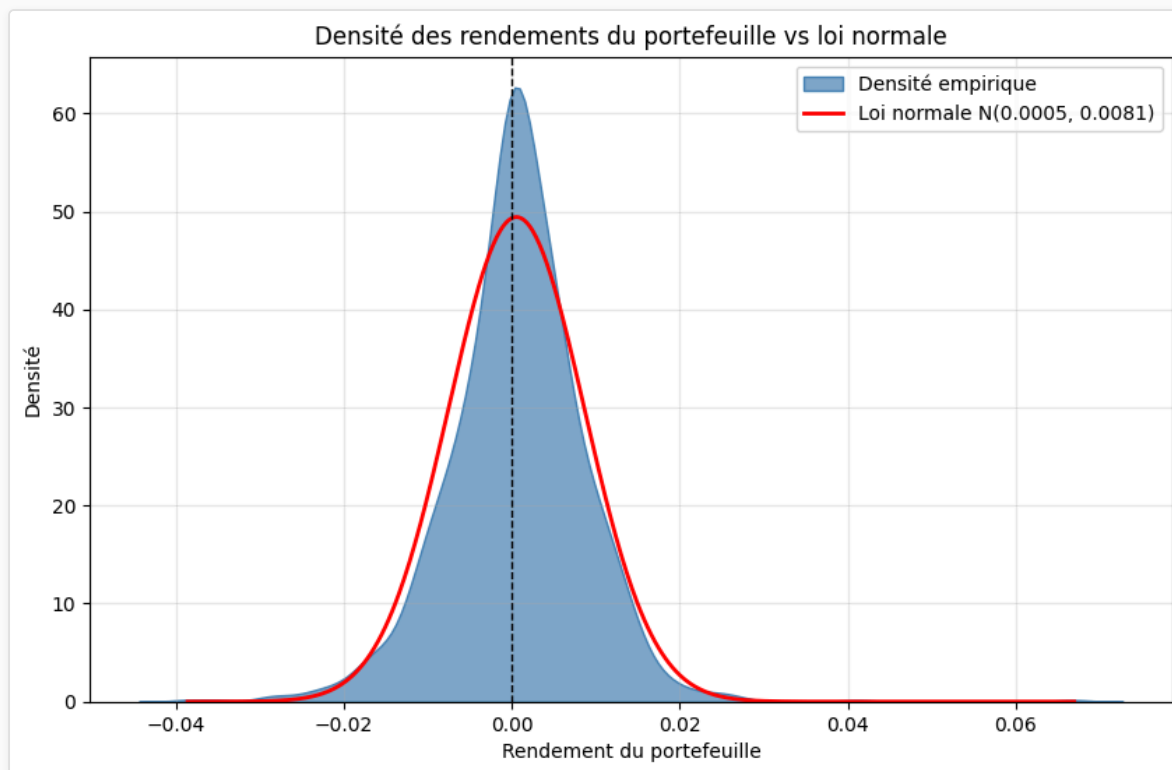
#parametres de La Loi normale
mu = portfolio_return.mean()
sigma = portfolio_return.std(ddof=1)

# Graphique
# superposition avec une loi normale de meme moyenne/ecart-type

# Grille pour la courbe de densité normale
x = np.linspace(portfolio_return.min(), portfolio_return.max(), 500)
plt.figure(figsize=(10, 6))
# Densité empirique du portefeuille
sns.kdeplot(portfolio_return, fill=True, color="steelblue", alpha=0.7, label="Densité empirique")
# Loi normale de même moyenne / écart-type
```

```
plt.plot(
    x,
    stats.norm.pdf(x, loc=mu, scale=sigma),
    color="red",
    linewidth=2,
    label=f"Loi normale N({mu:.4f}, {sigma:.4f})"
)
plt.axvline(0, color="black", linestyle="--", linewidth=1)
plt.title("Densité des rendements du portefeuille vs loi normale")
plt.xlabel("Rendement du portefeuille")
plt.ylabel("Densité")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Out [21]:



Analyse de la distribution des rendements

Le pic de la densité des rendements du portfolio est 12% plus élevé que le pic d'une loi normale de même écart-type et de même moyenne. De plus, la densité des rendements du portfolio présente des queues légèrement plus épaisses que la loi normale.

In [23]:

```
# Skewness (asymétrie)
skewness = stats.skew(portfolio_return)

# Kurtosis (excès de kurtosis, par défaut dans scipy = Fisher)
kurtosis = stats.kurtosis(portfolio_return)
print(f"Skewness du portefeuille : {skewness:.6f}")
print(f"Kurtosis du portefeuille : {kurtosis:.6f}")
```

Out [23]:

```
Skewness du portefeuille : 0.197325
Kurtosis du portefeuille : 5.072775
```

Le kurtosis est de 5,072775 soit 2 fois plus qu'une loi normale (généralement 3). Ce qui confirme les observations plus haut par rapports aux fat tails (les queues plus épaisses) et au pic central plus haut. Il en ressort que le portfolio est plutôt stable au quotidien avec des potentiels chocs extrêmes positifs comme négatifs plus important que ce que prévoit la normale. La skewness de 0.197325 indique qu'on devrait s'attendre plutôt à des chocs positifs.

Calcul de la VaR (3 méthodes) Pour un horizon 1 jour, confiance 95% et 99% :

```
In [25]:
#VaR paramétrique
from scipy.stats import norm

#on suppose que les rendements du portfolio suivent une loi normale

# parametres : moyenne mu, quantile z, et volatilité sigma (ecart-type)

def calcul_var(returns_, confiance):

    mu = np.mean(returns_)
    z = norm.ppf(1 - confiance)
    sigma = np.std(returns_, ddof = 1)

    var = -(mu + z*sigma)

    return var

print("VaR paramétrique confiance 95 % : ", calcul_var(portfolio_return, 0.95))
print("VaR paramétrique confiance 99 % : ", calcul_var(portfolio_return, 0.99))
```

```
Out [25]:
VaR paramétrique confiance 95 % :  0.01273026826293533
VaR paramétrique confiance 99 % :  0.018231108877245092
```

```
In [30]:
#var historique Le quantile empirique des rendements passés (pas d'hypothèse de distribution).

print(f"VaR historique confiance 95 % : {np.quantile(portfolio_return, 0.05)}" )
print(f"VaR historique confiance 99 % : {np.quantile(portfolio_return, 0.01)}" )
```

```
Out [30]:
VaR historique confiance 95 % : -0.012306893260166773
VaR historique confiance 99 % : -0.020830203473140595
```

```
In [32]:
#var Monte Carlo

# Paramètres de simulation
n_scenarios = 10_000
confiances = [0.95, 0.99]

# Moyenne et covariance des rendements des actifs
mu_actifs = returns.mean().values #pas le rendement du portefeuille mais celui des actifs
cov_actifs = returns.cov().values

# Simulation des rendements des actifs
simulations_actifs = np.random.multivariate_normal(
    mean=mu_actifs,
    cov=cov_actifs,
    size=n_scenarios
)

# Rendement simulé du portefeuille
simulations_portfolio = simulations_actifs.dot(poids)

for confiance in confiances:
    quantile = np.quantile(simulations_portfolio, 1 - confiance)
    var = -quantile

    print(
        f"VaR Monte Carlo à {confiance:.0%} : "
        f"{var: }"
    )
```

Out [32]:
VaR Monte Carlo à 95% : 0.01284682048312342
VaR Monte Carlo à 99% : 0.018043456025666562

VaR	95%	99%
VaR paramétrique	0.01273026826293533	0.018231108877245092
VaR historique	-0.012306893260166773	-0.020830203473140595
VaR Monte Carlo	0.01284682048312342	0.018043456025666562